Linux CedarE
开发指南

版本号: 2.1
发布日期: 2024.03.13

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2020.06.18	AWA0573	初稿
2.0	2023.05.10	AWA1908	添加背景介绍、相关术语、功能设计流程、一进二出介绍与使用等
2.1	2024.03.13	AWA1908	添加大小码流使用说明及 demo 案例、配置选项、FAQ

广州星翼电子科技有限公司zdyzlinux

目　录

1 概述 **1**
1.1 背景介绍 1
1.2 编写目的 1
1.3 适用范围 1
1.4 相关人员 1
1.5 代码位置及文件树 1

2 模块介绍 **3**
2.1 相关术语 3
2.2 详细设计 3
2.2.1 cedare 层级关系及组件 4
2.2.2 venc_comp_demo 7
2.2.3 recorder_demo 8

3 接口说明 **10**
3.1 recorder_create 10
3.2 recorder_destroy 10
3.3 recorder_setconfig 10
3.4 recorder_set_video_stream_callback 10
3.5 recorder_set_audio_stream_callback 11
3.6 recorder_set_yuv_buffer_callback 11
3.7 recorder_start_next_file 12
3.8 recorder_start 12
3.9 recorder_stop 12
3.10 recorder_pause 12
3.11 recorder_queue_yuv_buffer 13

4 数据结构说明 **14**
4.1 video_encoder_config_s 14
4.2 audio_encoder_config_s 14
4.3 yuv_config_s 15
4.4 muxer_config_s 15
4.5 recorder_video_stream_s 15
4.6 recorder_audio_stream_s 16
4.7 recorder_yuv_buf_s 16

5 大小码流使用说明及案例 **17**

6 menuconfig 配置选项 **19**

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7 FAQ 20

广州星翼电子科技有限公司zdyzlinux

插　图

图 2-1　整体编码框架 4
图 2-2　yuv_component 5
图 2-3　venc_component 6
图 2-4　venc_comp_demo input buffer 流转图 7
图 2-5　venc_comp_demo 组件流程 8
图 2-6　recorder_demo 组件流程 9
图 6-1 19
图 6-2 19

广州星翼电子科技有限公司zdyzlinux

1 概述

1.1 背景介绍

cedare 是一套轻量级的编码中间件，通过 JNI 接口直接给到上层使用，可以实现 android 原生 recorder 框架的基本功能，同时又可以更加灵活的提供客制化功能。由于相较于 android 原生框架简化了层级等，编码性能也有所提升。

由于 cedare 客制功能的灵活性，且根据用户需求重点开发了一进二出功能，以下流程都会带上两路编码进行介绍。

1.2 编写目的

详细描述 cedare 编码中间件针对编码和一进二出功能的设计流程，同时描述 cedare 对外 API 接口及相关的数据结构，指导基于编码中间件的开发和使用。

1.3 适用范围

T7/T507/T527 等各个芯片平台的 Android SDK 和 Linux SDK。

1.4 相关人员

编码相关产品的开发人员。

1.5 代码位置及文件树

cedare 代码存放于：frameworks/av/media/libcedare

关键文件 tree 如下：

```
.
├── base
├── components
│   ├── aenc_component.c
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
│   ├── component.c
│   ├── include
│   ├── mixer.c
│   ├── pcm.c
│   ├── venc_component.c
│   └── yuv_component.c
├── demo
│   ├── aenc_comp_demo
│   ├── recorder_demo
│   └── venc_comp_demo
├── muxer
│   ├── aac
│   ├── base
│   ├── include
│   ├── mp4
│   └── ts
└── recorder
    ├── recorder.c
    └── recorder.h
```

广州星翼电子科技有限公司zdyzlinux

2 模块介绍

2.1 相关术语

1. cedare：编码中间件，和 cedarx 同级
2. 一进二出：一种客制功能，支持同一输入源，输出两路不同码流，各路编码独立控制
3. yuv_component，用于管理 yuv input buffer 的组件
4. venc_component，编码组件，一进二出功能可以存在两个编码组件
5. aenc_component，audio 组件，用于 recorder 录制声音
6. muxer，可以把 video、audio stream 封装成 mp4 等文件形式

2.2 详细设计

cedare 提供编码场景中间件功能，输入 yuv 图像数据，yuv 图像数据经过主路和次路的图像编码，可选择视频码流数据与音频编码数据进行封装后写入文件。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2.1 cedare 层级关系及组件

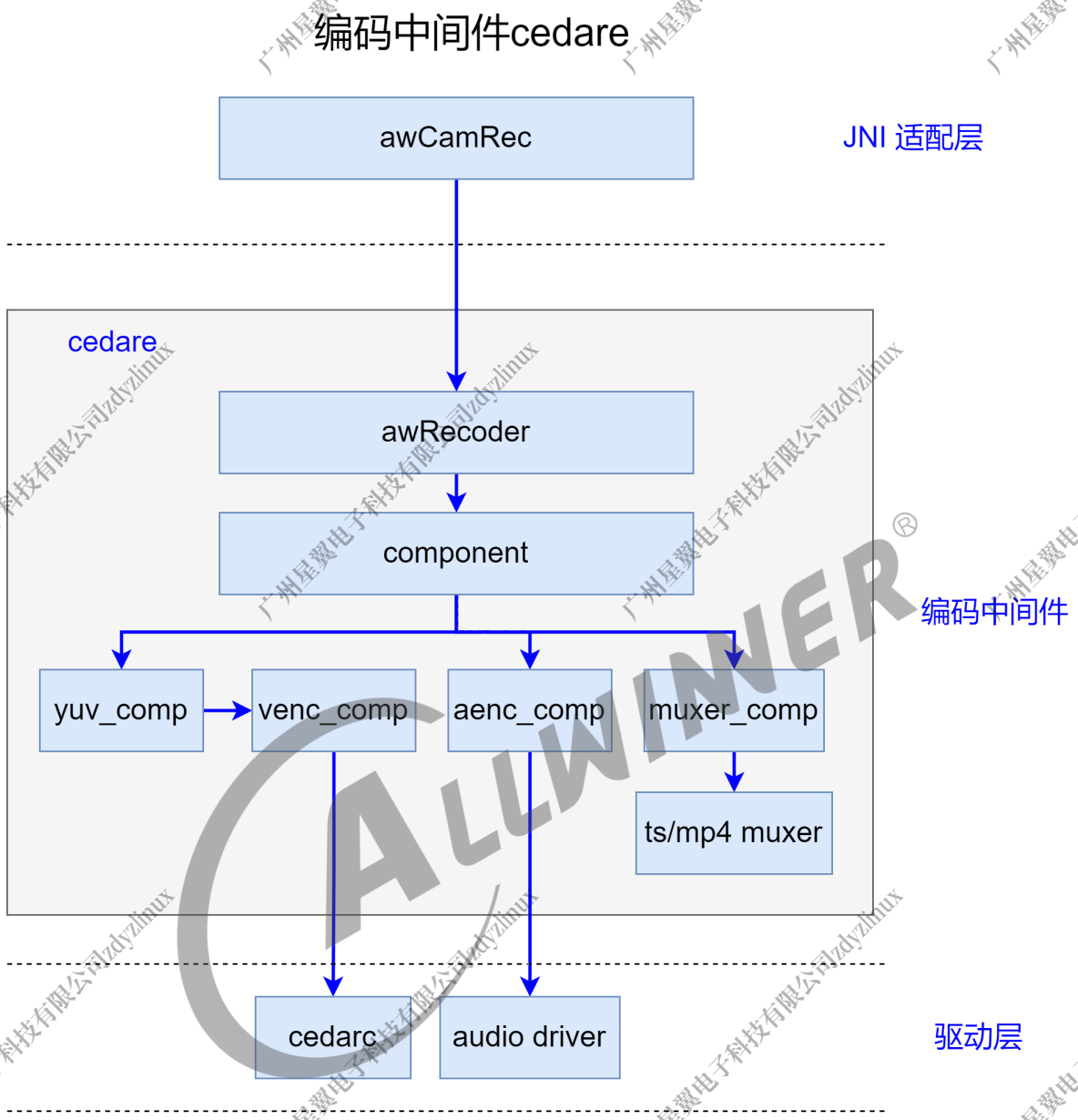

图 2-1: 整体编码框架

- AwCamRec 是 JNI 级 cedare 编码中间件适配层，向上直接给 app 提供接口
- cedare 编码中间件采用了组件设计模式，包含如下四种组件：

1. yuv_component：yuv 组件，用于管理 input buffer，输入可以是 csi，也可以是本地 yuv，输出组件为 venc_component，输出可以有多个；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

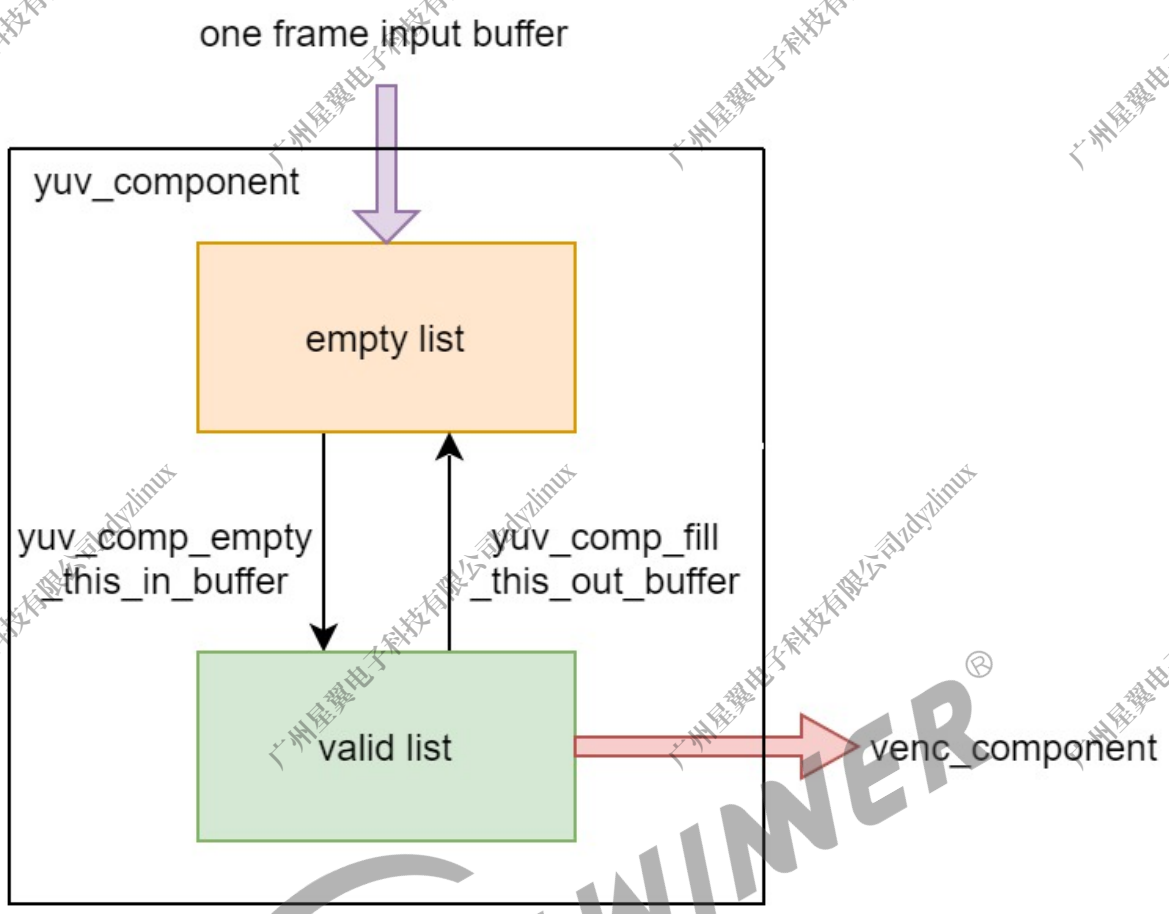

图 2-2: yuv_component

2. venc_component：编码组件，用于编码参数的初始化和客制化设置，从 yuv_component 获取 input buffer 后，传入 cedarc 驱动层进行编码，再获取 output buffer，可以选择 output buffer 回调给应用层做处理，或者直接传递给 muxer 封装成其他格式媒体文件。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

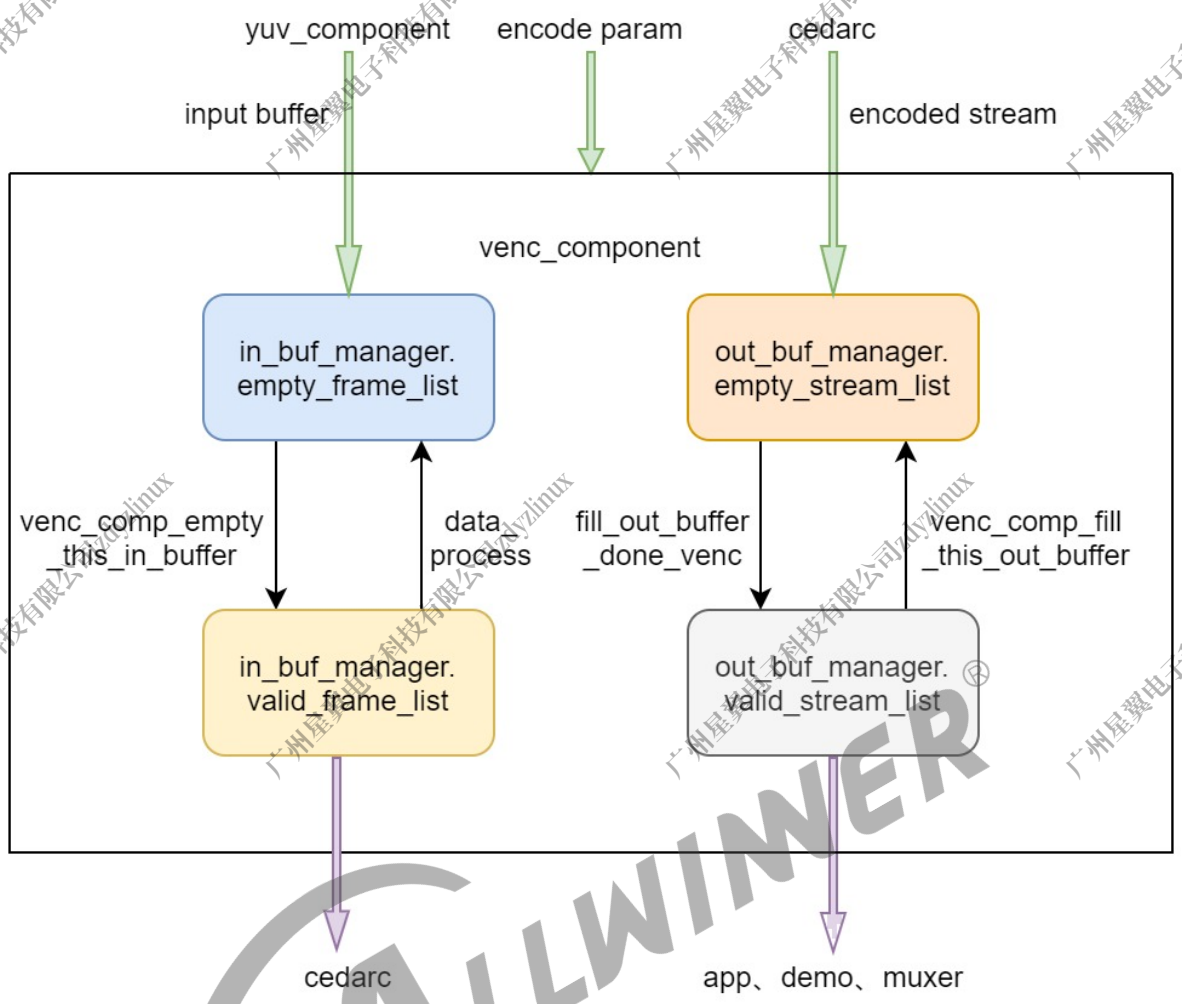

图 2-3: venc_component

3. aenc_component：音频组件，应用可选，aenc_component 创建后会进行音频的录制行为，并和 venc_component 编码码流进行同步，传递给 muxer 封装成其他格式媒体文件；
4. muxer_component：媒体文件封装组件，输入为 venc_component、aenc_component 编码音视频流，输出封装形式的 mp4、ts 格式等文件。

- 驱动层包含 cedarc、audio driver

1. cedarc，编码底层驱动，负责和硬件的交互，从 venc_component 获取 input buffer，输出编码后的码流
2. audio driver，音频底层驱动，在 cedare 中主要提供音频录制的硬件驱动功能

以下分列以 venc_comp_demo、recorder_demo 介绍 cedare 功能和流程。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2.2 venc_comp_demo

纯编码场景，包含两种 component：yuv_component、venc_component。

- input buffer 数据流转如下：

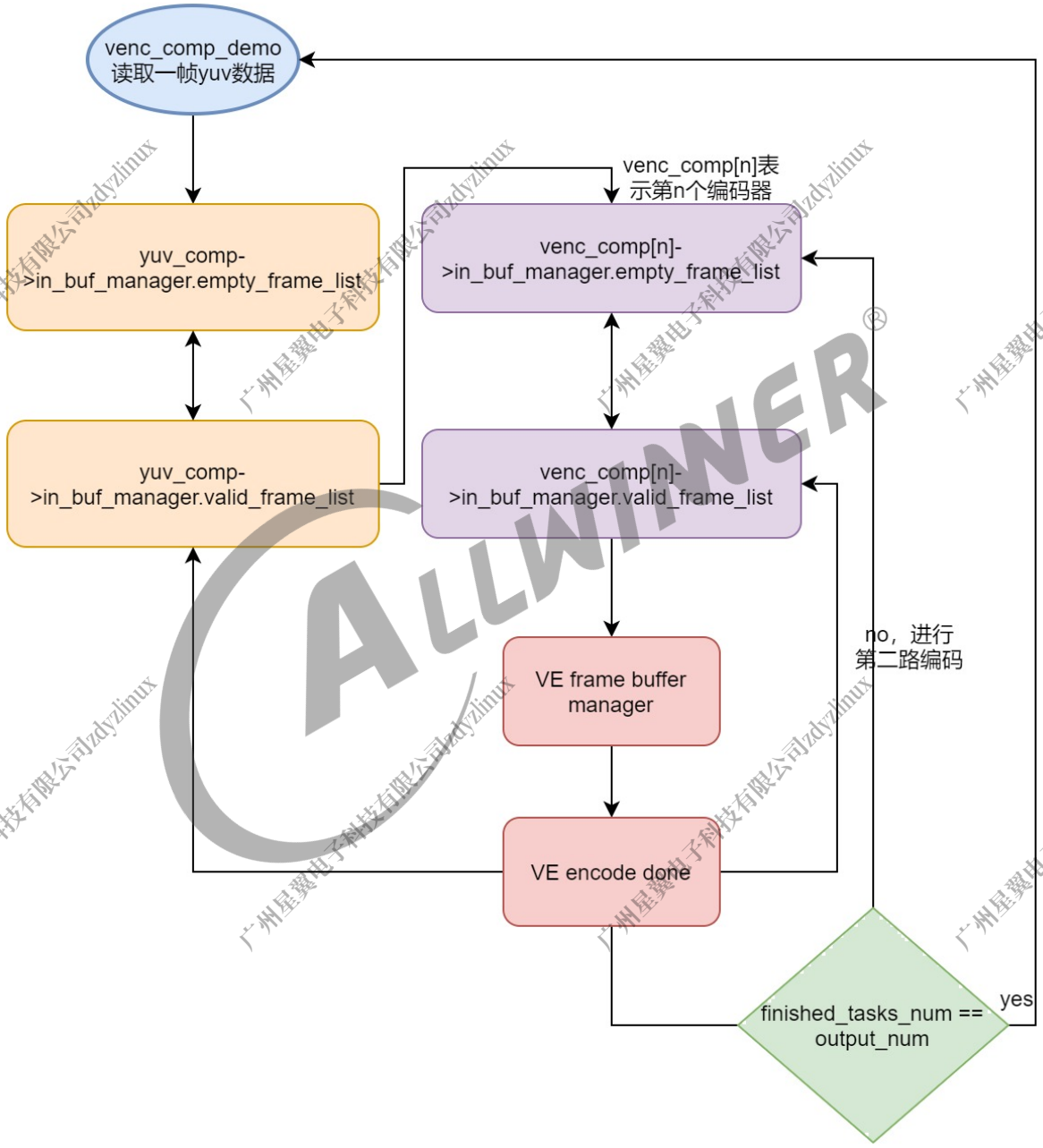

图 2-4: venc_comp_demo input buffer 流转图

- venc_comp_demo 模块组件流程图如下：

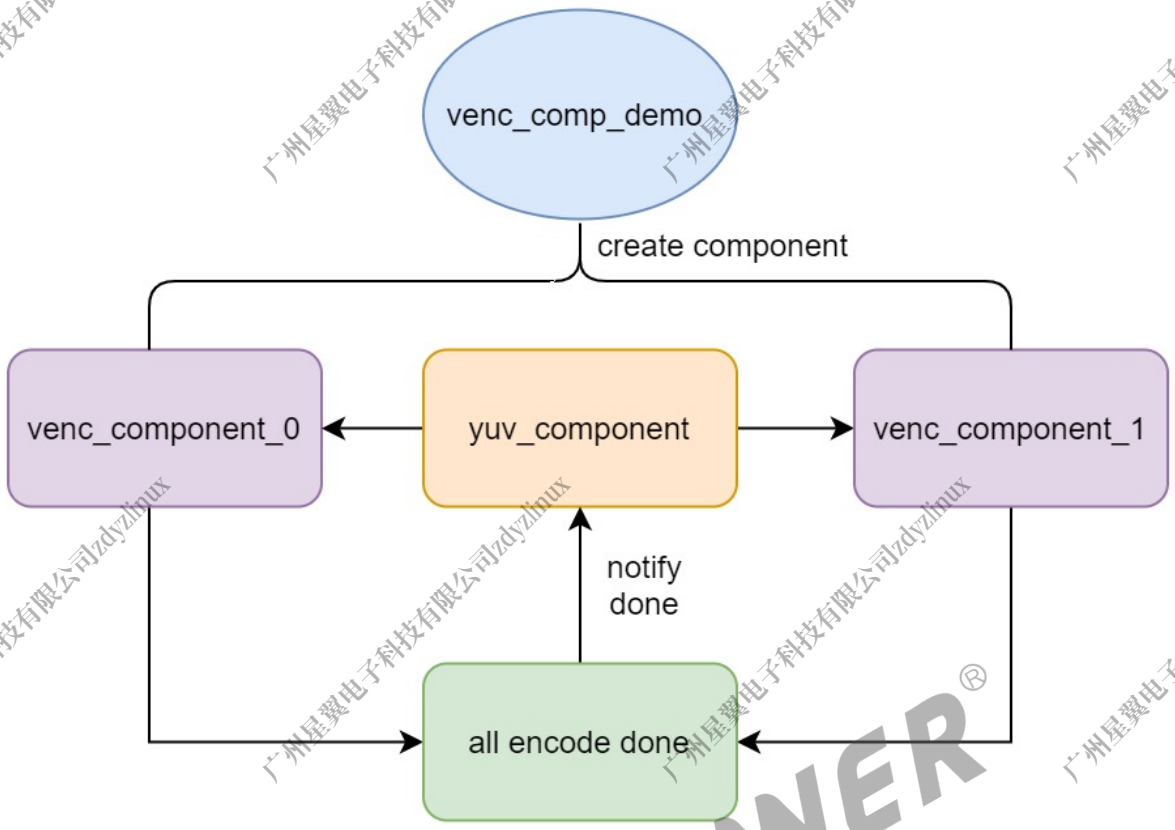

图 2-5: venc_comp_demo 组件流程

开启一进二出功能后，venc_comp_demo 包含三个组件：yuv_component、venc_component_0、venc_component_1。

- demo 获取一帧 yuv 数据后，通过 comp_empty_this_in_buffer 把数据给到 yuv_component 的 valid_frame_list，yuv_component 再根据 out_port_tunnel_info_list 中 venc 的数量，分别把数据给到相应的 venc_component，如一进二出功能，就会把同一 yuv 数据分别给到 venc_component_0 后 venc_component_1，两个编码组件分别完成编码后，通知 demo 可以读取下一帧 yuv 数据，demo 通过mDemoCtx->finished_tasks_num == mDemoCtx->output_num判断两路编码是否都已经完成了编码。

2.2.3 recorder_demo

recorder 组件除了 yuv_component、venc_component，还包含 aenc_component。

编码完成后，有两种模式：STREAM_OUTPUT_MODE_WRITE_TO_FILE，STREAM_OUTPUT_MODE_CALLBACK。

1. STREAM_OUTPUT_MODE_WRITE_TO_FILE，音视频数据编码完成后，通过 muxer 封装写入文件中。

2. STREAM_OUTPUT_MODE_CALLBACK，音视频数据回调给应用，让应用去客制处理数据，demo 例子音频数据是分别保存到不同文件中。

- 视频编码 input buffer 数据流转和 venc_comp_demo 基本一致
- muxer 方式组件流程图如下：

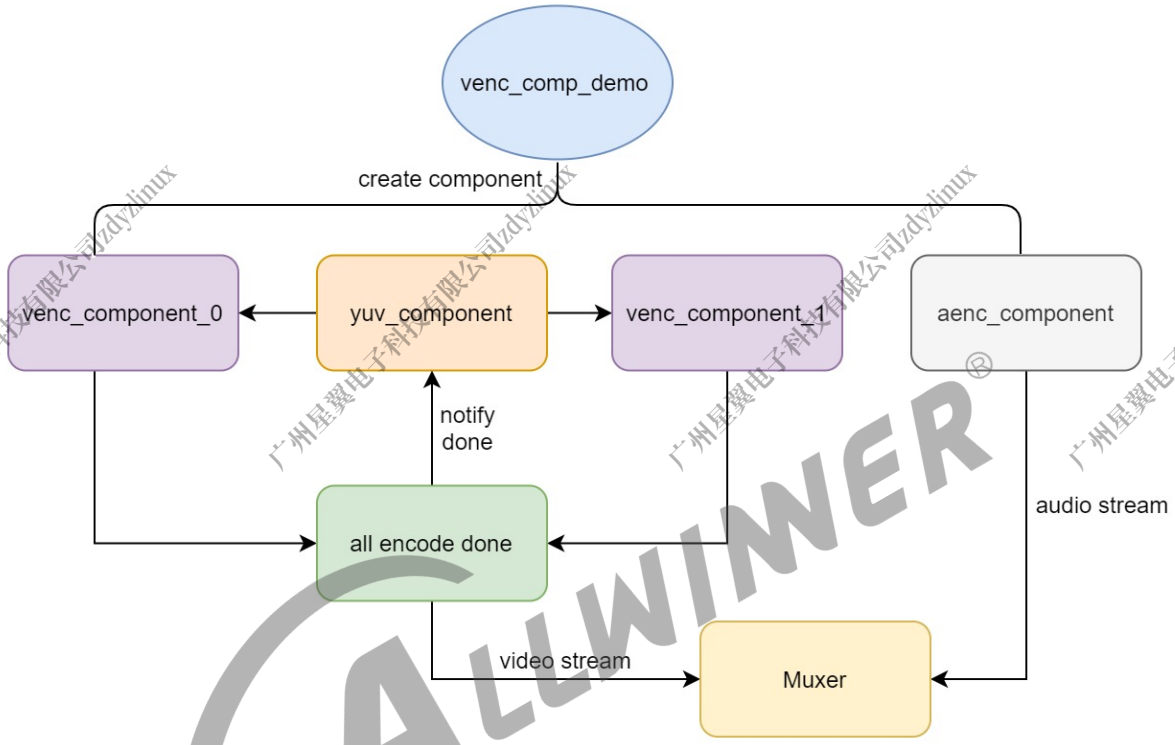

图 2-6: recorder_demo 组件流程

广州星翼电子科技有限公司zdyzlinux

3 接口说明

3.1 recorder_create

函数原型	recorder* recorder_create();
功能	创建录制器 recorder
参数	无
返回值	recorder 上下文指针
调用说明	无

3.2 recorder_destroy

函数原型	int recorder_destroy(recorder* recorder);
功能	销毁录制器 recorder
参数	recorder：recorder 上下文指针
返回值	无
调用说明	无

3.3 recorder_setconfig

函数原型	int recorder_setconfig(recorder* recorder, recorder_config* config);
功能	设置 recorder 的相关参数，包括 yuv 图像参数、视频编码通道参数、音频编码通道参数、封装器参数等
参数	recorder：recorder 上下文指针； config：参数集结构体指针
返回值	0：设置成功, 1: 设置失败
调用说明	无

3.4 recorder_set_video_stream_callback

函数原型	int recorder_set_video_stream_callback(recorder*recorder, video_stream_callback cb,

版权所有 © 珠海全志科技股份有限公司。保留一切权利

	int channel_id, void* privateData);
功能	设置视频编码码流数据的回调函数，recorder 通过回调函数把视频编码码流数据回调到 caller 层，caller 需要在此回调函数实现处理 video stream data 的操作。
参数	recorder：recorder 上下文指针; cb：回调函数 channel_id：视频编码通道 id，即主路编码或者子路编码的通道 id privateData：caller 层可设置的私有数据
返回值	0：设置成功, 1: 设置失败
调用说明	无

3.5 recorder_set_audio_stream_callback

函数原型	int recorder_set_audio_stream_callback(recorder* recorder, audio_stream_callback cb, void* privateData);
功能	设置音频编码码流数据的回调函数，recorder 通过回调函数把音频编码码流数据回调到 caller 层，caller 需要在此回调函数实现处理 audio stream data 的操作。
参数	recorder：recorder 上下文指针; cb：回调函数 privateData：caller 层可设置的私有数据
返回值	0：设置成功, 1: 设置失败
调用说明	无

3.6 recorder_set_yuv_buffer_callback

函数原型	int recorder_set_yuv_buffer_callback(recorder* recorder, yuv_buffer_callback cb, void* privateData);
功能	设置 yuv buffer 的回调函数，recorder 通过回调函数把已经用完的 yuv buffer 回调到 caller 层，caller 拿到此 yuv buffer 后可用于重新填写图像数据。
参数	recorder：recorder 上下文指针; cb：回调函数 privateData：caller 层可设置的私有数据
返回值	0：设置成功, 1: 设置失败
调用说明	无

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.7 recorder_start_next_file

函数原型	int recorder_start_next_file(recorder* recorder, char* file_name);
功能	设置文件路径，此文件用于写入封装数据（将视频和音频的码流数据进行封装）
参数	recorder：recorder 上下文指针； file_name：文件路径名称。
返回值	0：设置成功, 1: 设置失败
调用说明	caller 层需在调用 recorder_start 之前调用此函数进行设置文件路径；在后续的录制过程中，caller 可重新设置文件路径，如以 1 分钟为周期重复设置文件路径。

3.8 recorder_start

函数原型	int recorder_start(recorder* recorder);
功能	启动录制器 recorder
参数	recorder：recorder 上下文指针；
返回值	0：启动成功, 1: 启动失败
调用说明	无

3.9 recorder_stop

函数原型	int recorder_stop(recorder* recorder);
功能	停止录制器 recorder
参数	recorder：recorder 上下文指针；
返回值	0：停止成功, 1: 停止失败
调用说明	无

3.10 recorder_pause

函数原型	int recorder_pause(recorder* recorder);
功能	暂停录制器 recorder
参数	recorder：recorder 上下文指针；
返回值	0：暂停成功, 1: 暂停失败
调用说明	无

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.11 recorder_queue_yuv_buffer

函数原型	int recorder_queue_yuv_buffer(recorder* recorder, recorder_yuv_buf_s* yuv_buf);
功能	往 recorder 传送 yuv buffer
参数	recorder：recorder 上下文指针； yuv_buf：描述 yuv 图像数据的结构体指针
返回值	0：传送成功, 1: 传送失败
调用说明	caller 把图像数据往 buffer 里填充完成后，需调用此接口将 yuv buffer 传递到 recorder 进行编码；recorder 完成编码后，通过 recorder_set_yuv_buffer_callback 设置的回调函数将 yuv buffer 回调到 caller 层。

广州星翼电子科技有限公司zdyzlinux

4 数据结构说明

4.1 video_encoder_config_s

功能：用于初始化视频编码器的相关参数

属性	类型	描述
enable	int	1：开启此视频通道的编码；0：关闭
width	int	视频编码输出的图像宽度
height	int	视频编码输出的图像高度
frame_rate	int	帧率
bitrate	int	码率
I_frame_interval	int	I 帧帧间隔
qp_min	int	qp 的最小值，范围：1-51
qp_max	int	qp 的最大值，范围：1-51
vbr	int	码率控制模式； 1：VBR，可变码率 0：CBR，恒定码率
encoder_type	int	编码器类型，取如下枚举值： RECORDER_VENCODER_TYPE_H264 RECORDER_VENCODER_TYPE_H265 RECORDER_VENCODER_TYPE_JPEG
stream_output_mode	int	编码码流数据输出模式，取如下枚举值： STREAM_OUTPUT_MODE_WRITE_TO_FILE STREAM_OUTPUT_MODE_CALLBACK

4.2 audio_encoder_config_s

功能：用于初始化音频编码器的相关参数

属性	类型	描述
enable	int	1：开启此音频通道的编码；0：关闭
sample_rate	int	音频采样率，如 16000,441000 等
sample_bits	int	音频采样点的位深，如 16,32 等
channels	int	音频通道数，如 1、2 通道
bit_rate	int	码率

版权所有 © 珠海全志科技股份有限公司。保留一切权利

encoder_type	int	音频编码器类型，取如下枚举值： RECORDER_AENCODER_TYPE_AAC, RECORDER_AENCODER_TYPE_PCM, RECORDER_AENCODER_TYPE_MP3, RECORDER_AENCODER_TYPE_G711A,

4.3 yuv_config_s

功能：yuv 图像的属性参数

属性	类型	描述
width	int	yuv 图像的宽度
height	int	yuv 图像的高度
format	int	yuv 图像的像素格式，取如下枚举值： YUV_FORMAT_YUV420SP, YUV_FORMAT_YVU420SP, YUV_FORMAT_YUV420P, YUV_FORMAT_YVU420P, YUV_FORMAT_AFBC,

4.4 muxer_config_s

功能：用于初始化封装器的参数

属性	类型	描述
enable	int	1：开启封装器；0：关闭
muxer_type	int	封装器类型，取如下枚举值： RECORDER_MUXER_TYPE_MP4, RECORDER_MUXER_TYPE_TS,

4.5 recorder_video_stream_s

功能：：描述一笔视频编码后的码流数据的属性

属性	类型	描述
id	int	当前这笔码流的序列号
pts	int	时间戳
flag	int	相关标志

版权所有 © 珠海全志科技股份有限公司。保留一切权利

size0	int	第 1 段码流数据的长度
size1	int	第 2 段码流数据的长度
data0	int	第 1 段码流数据的 buffer 地址
data1	int	第 2 段码流数据的 buffer 地址
is_sps_pps_flag	int	当前码流是否为 sps/pps 数据的标志位

4.6 recorder_audio_stream_s

功能：描述一笔音频编码后的码流数据的属性

属性	类型	描述
id	int	当前这笔码流的序列号
pts	int	时间戳
size	int	码流数据的长度
data	int	码流数据的 buffer 地址

4.7 recorder_yuv_buf_s

功能：描述一帧 yuv 图像数据的属性

属性	类型	描述
id	int	当前 yuv 图像的序列号
pts	int	时间戳
phy_addr	int	yuv 图像数据的物理地址
vir_addr	int	yuv 图像数据的虚拟地址

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 大小码流使用说明及案例

1. 如需要 audio，可以参照 recorder_demo 的方式使用：recorder_config 结构体中video_encoder_config_s venc_config[ENCODER_CHANNEL_NUM];可以配置两个 venc，配置完主通道后，再配置次通道，关键点是venc_config[1].enable使能，如下例子：

```
//先配置完venc 0
 mRecorderConfig->venc_config[0].enable        = 1;
 mRecorderConfig->venc_config[0].scale_encoder_enable  = 1;
 mRecorderConfig->venc_config[0].width         = mRecorderConfig->yuv_config.width;
 mRecorderConfig->venc_config[0].height        = mRecorderConfig->yuv_config.height;
 ...
 ...
//再配置venc 1
 mRecorderConfig->venc_config[1].enable        = 1;
 mRecorderConfig->venc_config[0].scale_encoder_enable  = 0;
 mRecorderConfig->venc_config[1].width         = 1280;
 mRecorderConfig->venc_config[1].height        = 720;
 ...
 ...
```

venc 1 使能并配置后即可以实现同一 yuv 源，输出两路不同视频码流。

两路编码通道都可以独立配置自己的编码参数，配置方式和主路是一样的。开启第二路编码通道后，一些需要注意的配置如下：

- 设置第二路 audio 的回调函数：

```
video_channel = 1;
recorder_set_audio_stream_callback(mRecorder, audio_stream_data_cb,
  video_channel, (void*)mOutputAudioFile);
```

- 设置第二路码流回调函数：

```
video_channel = 1;//表示第二路
recorder_set_video_stream_callback(mRecorder, video_stream_data_cb,
  video_channel, (void*)mOutputVideoFile1);
```

- 启动第二路 recorder

```
ret = recorder_start(mRecorder, 1);
```

- 停止第二路 recorder

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
recorder_stop(mRecorder, 1);
```

如需编写自己的 demo/app，关于更详细的 recorder 使用方式，可以参照 demo/recorder_demo/recorder_demo.c 去进行参考和修改。

2. 单纯 video 编码可以使用 venc_comp_demo 的方式，一般是内部测试用。venc_comp_demo 提供了更灵活的 xml 配置方式，如下：

```
<demo>
<input name   = "input"
    path        = "/data/input.yuv"
    src_width  = "1920"
    src_height = "1080"
    frame_rate = "30"
    output_num = "2"
    >
</input>
<output name  = "output0"
    path        = "/data/output0.h264"
    dst_width  = "1920"
    dst_height = "1080"
    frame_rate = "30"
    bit_rate    = "6"
    qp_min      = "1"
    qp_max      = "51"
    interval    = "30"
    format      = "VENC_PIXEL_YUV420P">
</output>
<output name  = "output1"
    path        = "/data/output1.h264"
    dst_width  = "1080"
    dst_height = "720"
    frame_rate = "30"
    bit_rate    = "6"
    qp_min      = "1"
    qp_max      = "51"
    interval    = "30"
    format      = "VENC_PIXEL_YUV420P">
</output>
</demo>
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 menuconfig 配置选项

以 t527_linux_aiot 项目为例子，cedare 中间件代码是默认勾选的，source build/envsetup.sh后运行命令./build.sh buildroot_menuconfig，进入如下目录：

Target packages > allwinner platform private package select > libcedare

可以选择是否编译 cedare 代码，其中还可以选择编译 cedare 的 demo 进行测试。

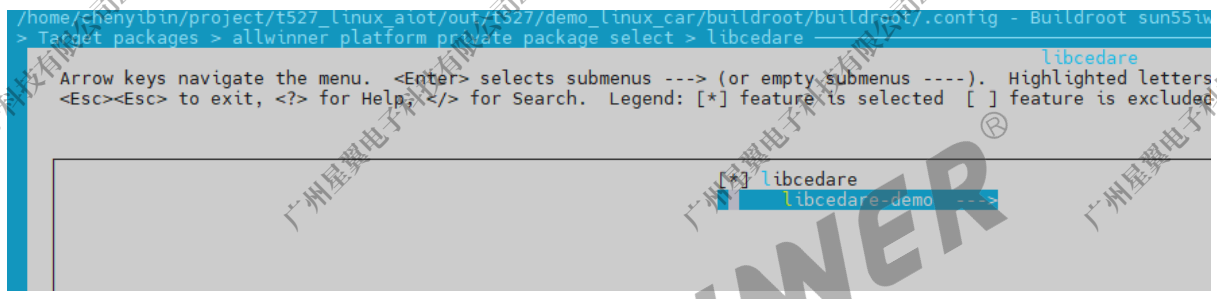

图 6-1

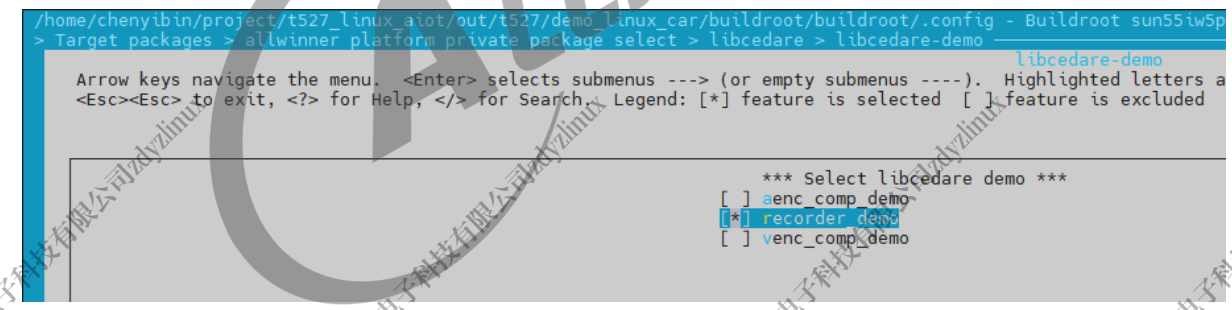

图 6-2

其他项目请参照相关具体配置要求设置。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7 FAQ

1. 如果 recorder 不想要声音，如何关闭?

关联到 audio 使能，代码逻辑中把它关掉即可mRecorderConfig->aenc_config.enable = 1;

2. 编码码流底部固定有绿色条状图，一般是什么原因导致?

底部有绿色条纹，一般是输入 buffer 传入底层，计算 buffer 大小的宽高没有做 16 对齐或者上层和底层的 Y、U 地址偏移不一致导致：

- yuv buffer 整体传入，申请 buffer 大小时，先把宽高做 16 对齐，再计算大小，如 yuv420 1920x1080，buffer size = 1920x1088(aligned 16)x3/2;
- yuv buffer Y、U 分开传入，底层申请的 buffer 计算已经有做 16 对齐，此时 addr_y = yuv buffer addr，addr_u = yuv buffer addr + 1920x1080。

3. 编码码流花屏或者有不规则条纹时隐时现，但是输入 yuv buffer 是正常的，如何分析?

遇到这种情况，可能是由于未刷 cache 导致，可以尝试调用刷 cache 接口测试是否正常，如刷新 Y、U buffer

```
int data_size = mRecorderConfig->yuv_config.width *\
    mRecorderConfig->yuv_config.height;
CdcMemFlushCache(mMemops, vir_addrY, data_size);
CdcMemFlushCache(mMemops, vir_addrC, data_size/2);
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2024 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。